

SIMULADOR DE ALGORITMOS DE SUBSTITUIÇÃO DE PÁGINAS (SASP)

Autor: Ricardo André - 2417200

Resumo

Este repositório apresenta um simulador de algoritmos de substituição de páginas em Java puro. O objetivo é demonstrar o comportamento de estratégias de gerenciamento de memória virtual, comparar o número de faltas de página e facilitar a análise de desempenho de cada algoritmo. A memória virtual é crítica em sistemas operacionais porque permite que processos acessem mais memória lógica do que a memória física disponível, e a seleção de páginas para substituição impacta diretamente a eficiência do sistema.

Introdução

A substituição de páginas surge quando a memória física está cheia e o sistema precisa escolher uma página existente para liberar espaço.

O simulador implementa quatro algoritmos clássicos:

- **FIFO (First-In, First-Out):** substitui a página mais antiga em memória.
- **LRU (Least Recently Used):** substitui a página que não foi usada por mais tempo.
- **Ótimo:** substitui a página cuja próxima referência ocorre mais tarde no futuro.
- **Relógio (Clock):** utiliza um ponteiro circular e um bit de uso para aproximar o comportamento do LRU com menor custo.

Esses algoritmos são representativos das diferentes abordagens para reduzir faltas de página e otimizar o desempenho de memória.

Metodologia (Explicação Técnica)

O código está dividido em duas partes principais:

1. Algoritmos.java

- Contém implementações modulares para FIFO, LRU, Ótimo e Relógio.
- Cada algoritmo recebe a sequência de páginas e a quantidade de quadros de memória.
- A classe utiliza estruturas de dados adequadas para cada estratégia:
 - `LinkedList` para FIFO, por sua ordem natural de chegada.
 - `ArrayList` com remoção baseada em posição para LRU, registrando recência de uso.
 - Pesquisa futura no vetor para Ótimo, garantindo a escolha correta da página a ser removida.
 - Vetor circular e bits de uso para o algoritmo Relógio, simulando um ponteiro de substituição.

2. Interface.java

- Cria uma interface Swing simples para entrada de dados e exibição dos resultados.
- Aceita sequência de páginas como texto e número de quadros como seletor.
- Gera uma visualização gráfica com barras coloridas em `JPanel` para comparar rapidamente as faltas de página de cada algoritmo.
- Apresenta também um resumo numérico das faltas de página para análise acadêmica.

A escolha das estruturas está alinhada com a natureza de cada algoritmo: FIFO usa `LinkedList` para manter a ordem de chegada, enquanto Ótimo exige leitura do futuro para tomar decisão ótima.

Como Executar

1. Abra um terminal na pasta do repositório.
2. Compile os arquivos Java:

```
javac src/Algoritmos.java src/Interface.java
```

3. Execute a interface gráfica:

```
java -cp src Interface
```

4. Insira a sequência de páginas separada por vírgulas, espaços ou ponto-e-vírgula.
5. Ajuste o número de quadros e clique em "Executar simulação".

Resultados Esperados

O simulador deve mostrar quantitativamente quantas faltas de página cada algoritmo produziu para a mesma carga de referência.

Os gráficos em barras ajudam a visualizar o comportamento relativo entre as estratégias.

- **FIFO** tende a ser simples, mas pode falhar em padrões de acesso repetidos.
- **LRU** costuma ser mais eficiente em cargas de trabalho com localidade temporal.
- **Ótimo** fornece referência teórica para o limite inferior de faltas, mas não é viável em sistemas reais porque exige conhecimento do futuro.
- **Relógio** oferece um compromisso prático entre custo de implementação e eficiência.

Essa comparação ajuda a escolher o algoritmo mais adequado para diferentes tipos de aplicações e cargas de trabalho.

Conclusão

O simulador demonstra que a eficiência teórica nem sempre é simples de aplicar na prática.

O algoritmo Ótimo é ideal em termos de número de faltas, mas sua implementação exige previsão de acessos futuros.

LRU e Relógio são escolhas realistas para sistemas operacionais porque aproximam bom desempenho sem exigir conhecimento futuro.

O hardware e a capacidade de processamento também influenciam o resultado: algoritmos com custo de gerenciamento mais alto podem reduzir faltas, mas aumentar o tempo de CPU.

Assim, a escolha do algoritmo deve equilibrar o número de faltas de página com a complexidade de implementação e o custo de execução.

Repositório

Link: <https://github.com/Dedezaao/SASP>